

✓ Grounding Techniques

Day 4 — Notebook 2 of 4 | Estimated Time: 30 minutes

What You'll Learn

- What grounding is and why it reduces hallucinations
- Prompt-based grounding: providing context the model must stay within
- Citation enforcement: requiring the model to reference its sources
- Fact-checking patterns: asking the model to verify its own claims
- Google Search grounding via the Gemini API

✓ Setup

```
%pip install -U -q "google-genai>=1.0.0"
```

```
!pip install google-auth==2.47.0
```

```
Requirement already satisfied: google-auth==2.47.0 in /usr/local/lib/python3.12/dist-packages (2.47.0)  
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (0.4.1)  
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages (from google-auth==2.47.0) (4.9.1)  
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0.2.1->google-auth==2.47.0) (0.6.1)
```

```
import sys, os  
import time  
  
import json  
import re  
from google import genai  
from google.genai import types  
from IPython.display import Markdown  
  
try:  
    from google.colab import auth  
    auth.authenticate_user()  
    print("✅ Authenticated via Colab")  
except ImportError:  
    print("✅ Using local credentials (gcloud auth)")  
  
client = genai.Client(  
    vertexai=True,  
    project="chanakya-476010",  
    location="us-central1",  
)  
MODEL_ID = "gemini-2.5-flash"  
print("✅ Ready!")
```

```
✅ Authenticated via Colab  
✅ Ready!
```

1. What Is Grounding?

Grounding means anchoring the model's responses to a specific, verifiable source of truth. Instead of relying on the model's parametric memory (training data), you supply the facts directly.

Without grounding: [User question] → LLM → Answer (may hallucinate)

With grounding: [User question] + [Verified context] → LLM → Answer (constrained to context)

Grounding strategies:

1. **Prompt-level grounding** — paste the source material directly into the prompt
2. **Citation enforcement** — require the model to cite every claim
3. **Self-verification** — ask the model to check its own output against the context
4. **Tool-based grounding** — use Google Search or a database to fetch real-time facts

✓ 2. Prompt-Level Grounding

The simplest form of grounding: **paste the source material** into the prompt and instruct the model to only use that material.

```
# WITHOUT grounding — model may hallucinate details
response_ungrounded = client.models.generate_content(
    model=MODEL_ID,
    contents="What are the main features of the company's Enterprise plan?",
)
print("🔴 UNGROUNDED:")
print(response_ungrounded.text)
print()
```

🔴 UNGROUNDED:

To give you the most accurate information, I need to know **which company's Enterprise plan you're interested in.**

Enterprise plans vary significantly between different software, SaaS providers, and service companies. However, I can give you a general overview of what Enterprise plans typically go beyond standard business plans by offering:

1. **Enhanced Security & Compliance:**
 - Single Sign-On (SSO):** Integration with corporate identity providers (SAML, OAuth, OpenID Connect) for seamless and secure access.
 - Advanced Access Controls:** Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), granular permissions.
 - Data Encryption:** Often offers higher standards or options for data encryption at rest and in transit.
 - Audit Logs & Activity Monitoring:** Detailed logs of all user actions and system events for compliance and security auditing.
 - Compliance Certifications:** Adherence to industry-specific regulations (e.g., HIPAA, GDPR, SOC 2 Type 2, ISO 27001, FIPS 140-2).
 - Data Residency:** Options to host data in specific geographic regions to meet regulatory requirements.
 - Penetration Testing & Vulnerability Scans:** Often allow or provide results from regular security assessments.
2. **Scalability & Performance:**
 - High User Limits (or Unlimited):** Designed to accommodate large numbers of users and teams.
 - Increased Storage & Data Capacity:** Significantly higher storage limits, API call limits, or data transfer allowances.
 - Dedicated Infrastructure:** In some cases, offers dedicated servers or private cloud instances for enhanced performance.
 - Higher Uptime Guarantees (SLA):** More stringent Service Level Agreements (SLAs) with financial penalties for downtime.
 - Global Content Delivery Networks (CDNs):** For faster content delivery to users worldwide.
3. **Management & Administration:**
 - Centralized Admin Console:** A unified dashboard for managing all users, teams, settings, and policies across the organization.
 - User Provisioning & Deprovisioning (SCIM):** Automated user management through integration with corporate directories.
 - Advanced Reporting & Analytics:** Deeper insights into usage, performance, security, and compliance.
 - Policy Enforcement:** Tools to set and enforce organizational policies, such as password complexity, session timeouts, and device management.
 - Team & Group Management:** Sophisticated features for organizing users into teams and managing their access.
4. **Integration & Customization:**
 - Robust API Access:** More extensive and higher-volume API access for custom integrations with existing enterprise systems.
 - Webhooks:** Real-time notifications for automated workflows.
 - Custom Branding/White-Labeling:** Options to customize the platform's appearance with company branding.
 - Custom Workflows & Automation:** Tools to tailor the software to specific business processes.
 - Developer Support:** Dedicated resources for integration and custom development.
5. **Dedicated Support & Services:**
 - Dedicated Account Manager (DAM):** A single point of contact for strategic planning, contract management, and overall account health.
 - Technical Account Manager (TAM) / Solution Architect:** Dedicated technical experts for implementation, optimization, and troubleshooting.
 - 24/7/365 Priority Support:** Faster response times and higher-tier support channels (phone, dedicated chat).
 - Onboarding & Training:** Customized onboarding programs, workshops, and ongoing training for large teams.
 - Professional Services:** Assistance with complex deployments, data migration, custom development, or specialized consulting.
 - Regular Business Reviews:** Periodic meetings to review performance, plan for future needs, and ensure success.
6. **Pricing & Contract Flexibility:**
 - Custom Pricing:** Tailored pricing models based on specific usage, number of users, features, and contract length.
 - Flexible Contract Terms:** Negotiable terms, including payment schedules, legal clauses, and Service Level Agreements.

To get specific features for your company, please tell me:

- The company name** (e.g., Salesforce, Microsoft, Zoom, Adobe, Atlassian, HubSpot, ServiceNow, etc.)
- The product or service** if the company offers multiple (e.g., "Salesforce Sales Cloud Enterprise Plan" or "Zoom Enterprise Plan").

Once I have that, I can provide much more precise details!

```
# WITH grounding — model is constrained to the provided document
```

```

product_docs = """
=== PRICING PLANS (Last updated: 2024-Q4) ===

STARTER PLAN ($0/month)
- Up to 3 users
- 5 GB storage
- Email support only
- No API access

PROFESSIONAL PLAN ($49/month per seat)
- Unlimited users
- 100 GB storage
- Priority email + chat support
- Full API access (10,000 calls/day)
- Custom integrations

ENTERPRISE PLAN (Custom pricing)
- Unlimited users and storage
- 24/7 phone + dedicated account manager
- Unlimited API access
- SSO and advanced security
- SLA: 99.99% uptime guarantee
- On-premise deployment option
"""

response_grounded = client.models.generate_content(
    model=MODEL_ID,
    contents=f"""Answer the question using ONLY the information in the document below.
If the answer is not in the document, say "This information is not available in the provided documentation."

<document>
{product_docs}
</document>

Question: What are the main features of the Enterprise plan?""",
)

print("🟢 GROUNDED:")
print(response_grounded.text)

```

```

🟢 GROUNDED:
The main features of the Enterprise plan are:
- Unlimited users and storage
- 24/7 phone + dedicated account manager
- Unlimited API access
- SSO and advanced security
- SLA: 99.99% uptime guarantee
- On-premise deployment option

```

✓ 3. Citation Enforcement

Require the model to **cite every claim** using inline references to the source material.

```

research_excerpt = """
Source: "The Science of Sleep" (Johnson et al., 2023, Sleep Medicine Reviews)

[Paragraph 1] Adults require between 7 and 9 hours of sleep per night for optimal health.
Consistently sleeping fewer than 6 hours is associated with a 48% increased risk of developing
heart disease and a 15% increased risk of stroke.

[Paragraph 2] REM sleep, which occurs primarily in the second half of the night, is critical
for memory consolidation and emotional regulation. Disruption of REM cycles is linked to
increased anxiety and impaired decision-making.

[Paragraph 3] Exposure to blue light from screens within 2 hours of bedtime suppresses
melatonin production by up to 22%, delaying sleep onset by an average of 40 minutes.
"""

response = client.models.generate_content(
    model=MODEL_ID,
    contents=f"""Summarize the key health implications of sleep patterns based on the document below.

REQUIREMENTS:
- Every factual claim must be followed by a citation in the format [Paragraph X]
- Do not add any information not present in the document

```

- If you cannot find support for a claim, do not make it

```
<document>
{research_excerpt}
</document>
```

```
Summarize with citations:""",
)
```

```
Markdown(response.text)
```

Adults require between 7 and 9 hours of sleep per night for optimal health [Paragraph 1]. Consistently sleeping fewer than 6 hours is associated with a 48% increased risk of developing heart disease and a 15% increased risk of stroke [Paragraph 1]. Disruption of REM sleep cycles is linked to increased anxiety and impaired decision-making [Paragraph 2]. Additionally, exposure to blue light from screens within 2 hours of bedtime can suppress melatonin production by up to 22%, delaying sleep onset by an average of 40 minutes [Paragraph 3].

4. The "Only Use the Context" Instruction Pattern

A powerful system prompt pattern for grounded Q&A applications.

```
GROUNDING_SYSTEM_PROMPT = """You are a helpful assistant that answers questions strictly based on the provided context.
```

```
Rules:
```

1. Only use information explicitly stated in the context
2. If the answer is not in the context, respond exactly: "I don't have enough information in the provided context to answer this"
3. Never infer, extrapolate, or use external knowledge
4. Quote directly when possible"""

```
faq_context = """
```

```
RETURN POLICY:
```

```
Items may be returned within 30 days of purchase with original receipt.
```

```
Electronics have a 15-day return window. Items must be in original packaging.
```

```
Sale items are final sale and cannot be returned.
```

```
SHIPPING:
```

```
Standard shipping takes 5-7 business days and costs $5.99.
```

```
Express shipping (2-day) is available for $14.99.
```

```
Free shipping on orders over $75.
```

```
"""
```

```
questions = [
```

```
    "Can I return a laptop I bought 20 days ago?",
```

```
    "How much does shipping cost for a $50 order?",
```

```
    "Do you offer international shipping?", # Not in context!
```

```
]
```

```
for question in questions:
```

```
    response = client.models.generate_content(
```

```
        model=MODEL_ID,
```

```
        contents=[
```

```
            types.Content(role="user", parts=[types.Part(text=f"Context:\n{faq_context}\n\nQuestion: {question}")]),
```

```
        ],
```

```
        config=types.GenerateContentConfig(
```

```
            system_instruction=GROUNDING_SYSTEM_PROMPT,
```

```
            ),
```

```
        )
```

```
        print(f"Q: {question}")
```

```
        print(f"A: {response.text.strip()}")
```

```
        print("-" * 60)
```

```
Q: Can I return a laptop I bought 20 days ago?
```

```
A: No, you cannot return a laptop bought 20 days ago because "Electronics have a 15-day return window."
```

```
Q: How much does shipping cost for a $50 order?
```

```
A: For a $50 order:
```

```
* Standard shipping costs $5.99.
```

```
* Express shipping costs $14.99.
```

```
Q: Do you offer international shipping?
```

```
A: I don't have enough information in the provided context to answer this.
```

5. Self-Verification Pattern (Chain-of-Verification)

Ask the model to **generate an answer, then verify it** against the source. This is a simple implementation of the Chain-of-Verification (CoVe) technique.

```
company_facts = """
TechCorp was founded in 2015 by Elena Rodriguez and Marcus Webb.
The company is headquartered in Austin, Texas.
TechCorp has 1,200 employees as of 2024.
Annual revenue in 2023 was $340 million.
The company went public in 2021 on the NASDAQ under ticker TCH.
"""

question = "Who founded TechCorp and where is it located?"

# Step 1: Generate an answer
initial_response = client.models.generate_content(
    model=MODEL_ID,
    contents=f"""Based on this document, answer: {question}\n\n{company_facts}""",
)
print("👉 Initial Answer:")
print(initial_response.text)
print()

# Step 2: Verify the answer against the source
verification_response = client.models.generate_content(
    model=MODEL_ID,
    contents=f"""Verify this answer against the source document.
For each claim in the answer, check whether it is:
- ✅ Supported by the document
- ❌ Contradicted by the document
- ⚠️ Not mentioned in the document

<source_document>
{company_facts}
</source_document>

<answer_to_verify>
{initial_response.text}
</answer_to_verify>

Verification: """,
)

print("🔍 Verification:")
Markdown(verification_response.text)
```

👉 Initial Answer:
TechCorp was founded by **Elena Rodriguez and Marcus Webb**, and it is located in **Austin, Texas**.

🔍 Verification:
Here's the verification of the answer against the source document:

- TechCorp was founded by **Elena Rodriguez and Marcus Webb**: ✅ Supported by the document
- it is located in **Austin, Texas**: ✅ Supported by the document

6. Google Search Grounding

Gemini supports **real-time grounding via Google Search**. When enabled, the model fetches current information from the web before answering. This is especially useful for time-sensitive questions.

```
# Google Search grounding - the model retrieves live web results
response = client.models.generate_content(
    model=MODEL_ID,
    contents="What is the current version of Python and when was it released?",
    config=types.GenerateContentConfig(
        tools=[types.Tool(google_search=types.GoogleSearch())],
    ),
)

print("Answer with Google Search grounding:")
print(response.text)
```

```
print()

# Check if grounding metadata was returned
if response.candidates[0].grounding_metadata:
    metadata = response.candidates[0].grounding_metadata
    print("\n 📄 Grounding Sources:")
    if hasattr(metadata, 'grounding_chunks') and metadata.grounding_chunks:
        for chunk in metadata.grounding_chunks:
            if hasattr(chunk, 'web') and chunk.web:
                print(f"   - {chunk.web.title}: {chunk.web.uri}")
    else:
        print("   (Search was performed – grounding chunks not exposed in this API tier)")
```

Answer with Google Search grounding:
The current stable version of Python is 3.14.4, which was released on April 7, 2026.

📄 Grounding Sources:

- python.org: <https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGZFg9wyJavD7vnD01BsDg-OVrd9m5GedQDc7hDoc0On>
- python.org: https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQH7k6_wLoL0BfOTErG-umNXf4WkMyPZcCT_0qPgYEdXo

🔗 Exercise 1: Build a Grounded FAQ Bot

Create a grounded Q&A assistant that answers questions from a document you define.

```
# Exercise 1: Grounded FAQ Bot
# TODO: Fill in your own document content below (any topic: a recipe, a policy, a product spec)

my_document = """
There are 10 different subjects that are being offered in the course. And the pattern for the examinations are 4 cycle tests ar
Cycle Test 1 - February 3rd Monday to February 4th Monday
Cycle Test 2 - April 3rd Monday to April 4th Monday
Cycle Test 3 - August 3rd Wednesday to August 4th Wednesday
Cycle Test 4 - November 1st Wednesday to November 2nd Wednesday

The 10 different subjects are condensed into 6 days with grouping of subjects and hence only few dates are available.

"""

# TODO: Write 3 questions – 2 that are answerable from the document, 1 that is not
test_questions = [
    "For the year 2018 , when are the actual dates for Cycle Test 2",
    "In the year 2019 when did the Cycle Test 4 exams happen",
    "Does the term exam start on June 20th for any year with this given information",
]

# TODO: Use the grounded system prompt pattern to build the FAQ bot
# Check that the model refuses to answer the out-of-scope question

for question in test_questions:
    response = client.models.generate_content(
        model=MODEL_ID,
        contents=question,
        config=types.GenerateContentConfig(
            system_instruction=my_document,
        )
    )
    print(response.text)
```

Let's break down Cycle Test 2 for 2018:

* **Cycle Test 2:** April 3rd Monday to April 4th Monday

First, we need to find the dates for April 2018:

* April 1, 2018, was a Sunday.
 * Therefore, the 1st Monday of April 2018 was April 2nd.
 * The 2nd Monday of April 2018 was April 9th.
 * The **3rd Monday of April 2018 was April 16th**.
 * The **4th Monday of April 2018 was April 23rd**.

So, for the year 2018, Cycle Test 2 is from **April 16th to April 23rd, 2018**.
 Let's break down the timing for Cycle Test 4 in 2019:

The pattern for Cycle Test 4 is: **November 1st Wednesday to November 2nd Wednesday**

```
1. **Find November 1st, 2019:** November 1st, 2019 was a **Friday**.  
2. **Determine the 1st Wednesday of November 2019:**  
  * Nov 1 (Fri)  
  * Nov 2 (Sat)  
  * Nov 3 (Sun)  
  * Nov 4 (Mon)  
  * Nov 5 (Tue)  
  * **Nov 6 (Wed)** - This is the 1st Wednesday of November 2019.  
3. **Determine the 2nd Wednesday of November 2019:**  
  * Nov 6 (Wed) + 7 days = **Nov 13 (Wed)** - This is the 2nd Wednesday of November 2019.
```

So, in 2019, the Cycle Test 4 exams happened from **November 6th to November 13th**.
No, based *only* on the information provided, we **cannot determine** if a term exam starts on June 20th.

The details you've given specify the schedule for the **four cycle tests**, but **not** for the two term exams. To answer your q

Start coding or [generate](#) with AI.

Exercise 2: Citation Enforcement

Take the text below and ask the model to summarize it with mandatory inline citations.

```
# Exercise 2: Citation enforcement

climate_excerpt = """
[Section A] Global average temperatures have risen by approximately 1.1°C since the pre-industrial period (1850-1900).
The last decade (2011-2020) was the warmest on record.

[Section B] Sea levels are rising at an accelerating rate. The global mean sea level rose by 20 cm
between 1901 and 2018. The current rate of rise is 3.7 mm per year, more than double the rate
observed in the 20th century.

[Section C] Extreme weather events are becoming more frequent and intense. Heatwaves that previously
occurred once every 50 years now occur once every 10 years due to climate change.
"""

# TODO: Write a prompt that:
# 1. Asks for a 3-bullet-point summary of the key climate facts
# 2. Requires every bullet to include a citation to the section it came from
# 3. Prohibits adding any information not in the text

prompt = f"""Give a 3 bullet point summary of the key climate facts. Have every bullet to have the citation it came from and
Mainly , it should come from here only
<source_document>
{climate_excerpt}
</source_document>
""" # TODO: Write your prompt here

response = client.models.generate_content(model=MODEL_ID, contents=prompt)
Markdown(response.text)
```

Here is a 3-bullet point summary of the key climate facts from the provided document:

- Global average temperatures have increased by approximately 1.1°C since the pre-industrial period (1850-1900), with the last decade (2011-2020) being the warmest on record. [Section A]
- Sea levels are rising at an accelerating rate, with the global mean sea level increasing by 20 cm between 1901 and 2018, and the current rate of rise being 3.7 mm per year. [Section B]
- Extreme weather events are becoming more frequent and intense; for example, heatwaves that previously occurred once every 50 years now happen once every 10 years due to climate change. [Section C]

Key Takeaways

Technique	When to Use	Limitation
Prompt-level grounding	Short documents, single turn	Limited by context window
Citation enforcement	Reports, summaries, research	Model may still fabricate if not careful
Self-verification	High-stakes responses	Adds latency and cost
Google Search grounding	Real-time or recent information	Requires API access, internet connectivity

Further Reading

Resource	Type	Link
Grounding with Google Search	Docs	ai.google.dev/gemini-api/docs/grounding
Chain-of-Verification (CoVe)	Paper	arxiv.org/abs/2309.11495
RAG for Grounding (preview)	Coming in Day 7	Retrieval-Augmented Generation

Next up: [03_Generation_Control_Parameters.ipynb](#)